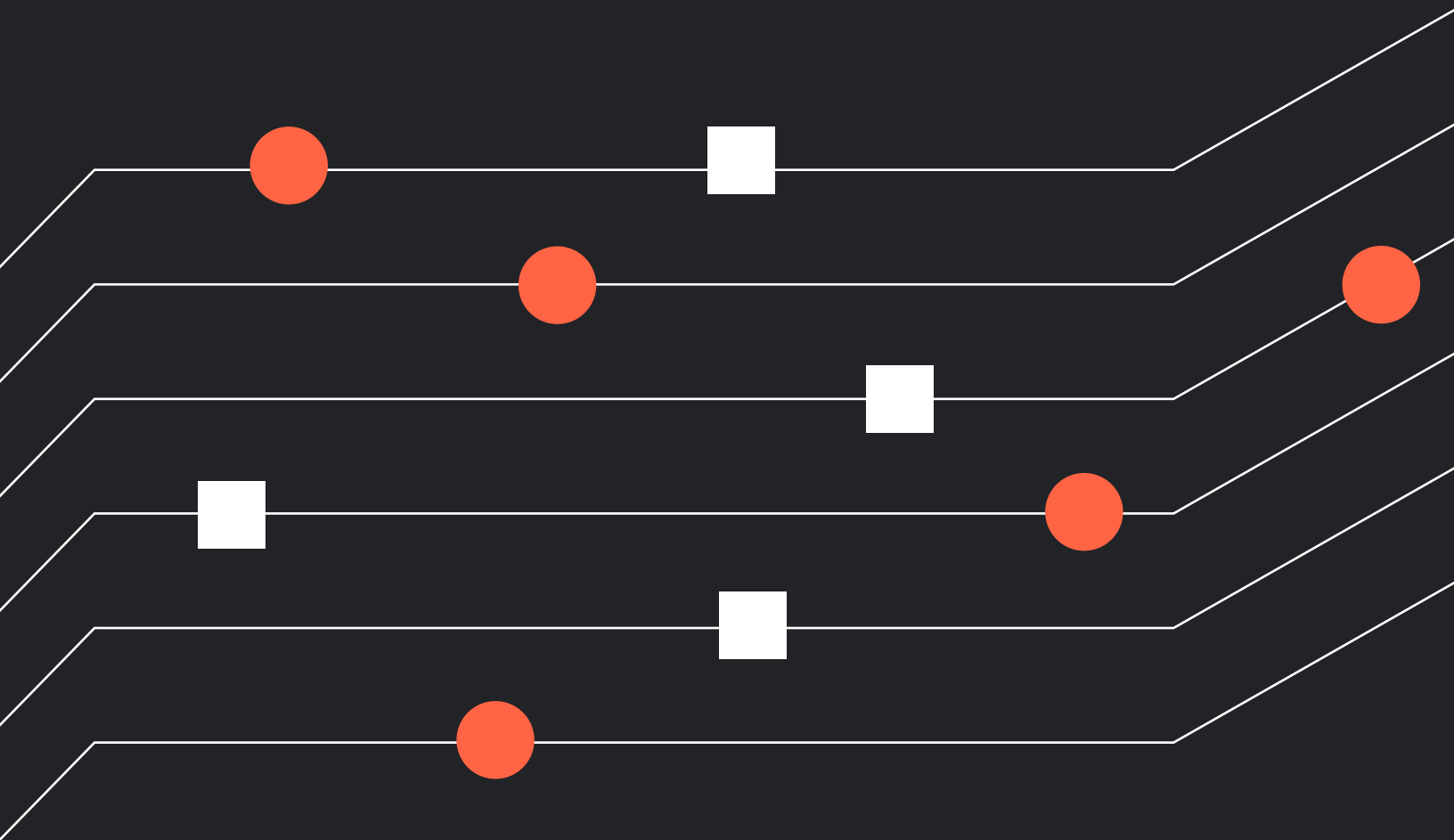


6 trends shaping the future of AI-driven development

What we've learned from
our community of 20K
engineering leaders.



Introduction

AI is no longer a fringe tool for automating code snippets; it's becoming the backbone of modern software engineering. In 2025, we're witnessing a fundamental shift from isolated AI features to intelligent systems that drive decisions, execute actions, and reshape how teams build and ship software. Yet, while AI-powered code generation has dominated headlines, the transformation is happening elsewhere: in how teams manage knowledge, orchestrate workflows, modernize infrastructure, and build trust in AI.

This guide goes beyond the hype to explore six emerging trends defining the future of AI-driven software engineering. Based on insights from leading voices in the Dev Interrupted community and research from LinearB, we unpack the opportunities, challenges, and practical steps engineering leaders must take today to thrive in tomorrow's AI-first landscape.

Whether you're just beginning to experiment with AI tools or scaling agentic systems across your SDLC, this is your roadmap for navigating the next evolution of software development.

-
- 1 Unified knowledge is the new productivity fuel
 - 2 Modernizing infrastructure for agentic workflows
 - 3 Using governance as an AI enabler
 - 4 From tools to orchestrated systems
 - 5 Recalibrating for developer-agent collaboration
 - 6 Feedback loops make or break AI adoption
 - 7 Checklist: Prepare for AI-driven software development

Unified knowledge is the new productivity fuel

AI agents need context—code, tickets, documentation, team norms, and architecture decisions to operate effectively. But most organizations struggle with fragmented knowledge, outdated documentation, and inaccessible tribal wisdom. Without curated, high-quality data, even the best models falter.



The challenge

- Critical information is often fragmented across tools or trapped in tribal knowledge
- AI agents may act on incomplete or outdated data
- Data silos hinder access to essential context
- Inconsistent or missing documentation slows down AI-assisted onboarding and decision-making



What you can do today

- Establish data hygiene practices across your engineering organization
- Create and maintain high-quality training datasets for your AI models
- Work toward centralizing knowledge that's currently fragmented or exists only as tribal knowledge
- Invest in documentation automation to reduce the burden of keeping information current



The “good data in, good data out” principle remains fundamental to effective AI implementation, particularly as organizations move toward agentic systems in software engineering. Brandon Jung, VP of Ecosystem at Tabnine, emphasizes that data provenance and quality are essential for building trust in AI-powered development tools: “We’ve always known that AI is good data in, good data out, bad data in, bad data out. So from that aspect, that’s not changed just because it’s generative.”

This insight is especially critical as engineering teams consider what code their developers use as inspiration and reference material. For organizations implementing AI coding tools, Jung recommends creating “golden repositories” of trusted, high-quality code. These centralized, vetted knowledge sources are the foundation for effective AI implementation, ensuring that AI agents learn from your best practices rather than reinforcing poor patterns.

Without this centralized approach to knowledge management, organizations risk introducing inconsistencies and security vulnerabilities as AI adoption scales. Data quality and unified knowledge repositories become more critical as teams apply AI to more specialized codebases, including legacy systems.

“Know your data, there’s no two ways about this. We’ve known this forever... It’s the lifeblood of what you got,” Jung concludes, reinforcing that high-quality, centralized knowledge isn’t just a prerequisite for AI; it’s the essential fuel that powers transformative implementations.



Brandon Jung

VP of Ecosystem



Modernizing infrastructure for agentic workflows

Most software teams still use outdated tooling and infrastructure practices.

Agentic AI requires new layers like agent observability, sandboxed testing, AI-native CI/CD, and zero-touch deployment pipelines.



The challenge

- Agentic AI requires tooling that's not yet widespread
- Many teams rely on outdated, manual ops practices with little automation or self-service capabilities
- Companies still depend on ticket-based workflows (Jira, ServiceNow) that aren't scaling for AI adoption
- Infrastructure burdens developers with tasks they aren't trained for



What you can do today

- Build a dedicated platform engineering function to abstract complexity
- Enable developers to ship code without infrastructure overhead
- Invest in mature pipelines and test coverage to ensure agentic AI adoption improves velocity without amplifying risk
- Prioritize self-service infrastructure to reduce dependencies on specialized teams



The widening skills gap in development operations presents a critical challenge for organizations embracing agentic AI. As Cory O'Daniel, CEO of MassDriver, notes, "The amount of operations experts or expertise that we have as a community is going down year over year because we're getting so many developers out of boot camps and out of colleges, and we're not teaching them the cloud and operations."

This disconnect is particularly concerning as AI integration demands even more infrastructure sophistication, such as agent observability, sandboxed testing environments, and AI-native CI/CD pipelines that most teams lack.

By abstracting this growing complexity, platform engineering emerges as the essential bridge to agentic workflows. Rather than forcing developers to become experts in everything from cloud configuration to security compliance, platform teams build products that create self-service infrastructure experiences.

According to O'Daniel, you can measure success by asking two questions: "Are developers happy with the way things work? Are the tools solving real problems for them?" Elite engineering organizations prioritize these questions, allowing new hires to deploy code on their first day and measuring how quickly developers can move from writing code to seeing it in production. This focus on developer experience creates the foundation for AI agents to operate effectively, removing the friction points and technical debt that would otherwise hinder AI adoption and scaling.



Cory O'Daniel

CEO



Using governance as an AI enabler

As agents begin to take action—merging PRs, triggering deployments, or responding to production errors—oversight becomes mission-critical. The probabilistic nature of LLMs makes explainability and auditability difficult but essential.



The challenge

- Autonomous agents making decisions (merging PRs, deploying hotfixes) raise serious concerns about safety, auditability, and accountability
- LLMs are inherently probabilistic, yet teams ask them to make deterministic decisions in high-stakes environments
- The “black box” nature of AI systems makes it hard to trace reasoning or ensure explainability
- Without clear audit trails or trustworthy data sources, teams can’t reliably validate agent decisions



What you can do today

- Build better audit trails and trustworthy data sources for validating agent decisions
- Implement human-in-the-loop mechanisms with robust policy enforcement
- Design governance frameworks that create guardrails, showing developers the easiest path to follow
- Don’t “shift left” compliance responsibility onto developers without giving them automated tools to take action



As agentic AI systems become more autonomous in software engineering workflows, trust emerges as the critical foundation for successful implementation. “Trust is the only thing that is going to matter at a foundational level,” explains Brooke Hartley Moy, CEO of Infactory. “For AI to have any meaningful impact on the enterprise and society at large, people have to believe that the outputs are accurate and reliable, and that they’re based on outcomes that ultimately have net benefit as opposed to risks and dangers embedded in the system.”

This perspective reinforces why governance should be viewed not as a barrier but as an enabler: creating the necessary trust framework for AI adoption to scale. The inherently probabilistic nature of LLMs presents a fundamental challenge when teams apply these models to deterministic business decisions.

As Moy notes, “LLMs are this really magical technology... but they also aren’t the end-all-be-all solution or silver bullet that people have come to treat them as.” Engineering leaders must address this gap by implementing complementary governance techniques that pull back the black box as much as possible. This includes establishing clear audit trails, implementing

human-in-the-loop validation mechanisms, and building explainability features that help users understand the reasoning behind AI outputs.

As we move toward what Moy describes as a “much more agentic autonomous view of the world where it’s not just about search and retrieval, it’s about actual action,” these governance frameworks become essential enablers that allow organizations to scale AI adoption while maintaining accountability, trust, and safety.



Brooke Hartley Moy

CEO



From tools to orchestrated systems

AI tools today are often bolted onto workflows rather than truly integrated. However, fundamental transformation will come from orchestration and systems where agents communicate, delegate, and reason across stages of the SDLC.

Until protocols mature, teams must build bespoke integration layers. Start by identifying repeatable, high-leverage tasks with clean APIs and consistent usage patterns. The products that solve this more comprehensively will come later, creating an opportunity for those who can build these bridges today.



The challenge

- Most developer environments lack native support for autonomy or inter-agent communication
- Existing infrastructures often treat tools as isolated functions, not interconnected workflows
- No standardized protocol has emerged for agent communication, forcing teams to rebuild integrations for each use case



What you can do today

- Focus on low-hanging fruit in your workflows: tasks that consume large amounts of time have strong API integrations and standardized usage patterns
- Build custom integration layers that connect your most critical systems
- Prepare for a period of bespoke solutions until standards emerge
- Identify repeatable, high-leverage tasks with clean APIs and consistent usage patterns



Agentic AI represents a fundamental evolution beyond traditional automation. While today's AI tools often function as isolated assistants bolted onto existing workflows, true transformation emerges when these agents begin communicating and reasoning across the entire software development lifecycle. As Amir Behbehani, Chief AI Engineer at Memra, explains, this shift represents a move from humans being "in the middle of workflows" to being "at the top of workflows," providing high-level direction. At the same time, AI handles complex reasoning and execution processes.

This orchestration layer is developing from the bottom up rather than top down. According to Behbehani, "The foundational layer comes about first, then the RAG layer, then the short-term memory layer, and then these agentic frameworks." This evolution creates a new paradigm where engineers shift from a "carpenter" mindset focused on execution to an "architect" mindset, designing the systems that build systems. The challenge for engineering leaders is clear: prepare for AI that doesn't just augment specific development tasks but orchestrates entire workflows across previously siloed functions.



Amir Behbehani

Chief AI Engineer

memra

Recalibrating for developer-agent collaboration

Developers are accustomed to using AI tools rather than working alongside autonomous agents. But that's changing. In 2025, agents will increasingly initiate tasks and require human validation, not the other way around.



The challenge

- Developers aren't used to collaborating with agents as co-workers, especially ones that take initiative
- Autonomous agents may act misaligned with team goals, causing friction and inefficiency
- Cognitive burden increases when developers override agent decisions
- Automation without guardrails leads to review overload, especially with bot-generated PRs



What you can do today

- On the probabilistic side: Work to change organizational norms to accept agents as valid contributors
- On the deterministic side: Automate guardrails to reduce mental load
- Don't force AI compliance to "shift left" onto developers without actionable, automated oversight
- Establish clear interaction patterns between developers and agents
- Design AI experiences that fit naturally into existing developer workflows



As autonomous agents become integral to development workflows, the distinction Tara Hernandez, VP of Developer Productivity at MongoDB, makes becomes increasingly relevant: “Automation does not equal autonomy. If you have automation that goes nuts, it probably is doing the reverse of what you would hope.” This insight directly addresses the emerging challenge of recalibrating how developers work alongside AI agents that increasingly take initiative rather than merely responding to commands. The risk of misalignment between agent actions and team goals creates friction that can disrupt developer flow and increase cognitive burden if not properly managed.

Hernandez’s approach to this challenge offers a compelling framework for developer-agent collaboration: “Our job is to make the golden path the easy path, and to make it so that you will naturally go the direction we want you to go, because that’s the easiest way to do it.” Applied to agent collaboration, this means implementing automated guardrails that naturally guide the interaction between developers and AI systems without adding complexity.

“You cannot expect the developers to keep a thousand things at the forefront of their minds while doing their day-to-day work. It’s just not going to work,” Hernandez emphasizes.

The most effective guardrails are almost invisible, making the compliant path easy to follow. This approach addresses the core concern of how to reduce friction and mental overhead as autonomous agents become more integrated into development workflows, allowing developers to maintain focus on their core work while the system naturally guides agent behavior toward alignment with team goals.



Tara Hernandez

VP of Developer Productivity



Feedback loops make or break AI adoption

AI-generated output is only as good as the feedback that follows. Most teams lack the structured, signal-rich environments to refine agentic behavior over time. Metrics are often noisy, inconsistent, or misaligned with business value.



The challenge

- It's hard to measure the quality of agentic decisions beyond short-term metrics like velocity or PRs per engineer
- More AI-generated code increases the need for human feedback: code review, validation, and evaluation
- Most teams lack robust, signal-rich environments that agents need to learn and improve
- Misaligned metrics or noisy alerts erode trust and reduce the quality of feedback loops



What you can do today

- Standardize metrics definitions and ownership to improve signal clarity
- Use proactive, data-driven rituals to institutionalize feedback loops
- Implement monthly metrics meetings, developer coaching, and cross-functional post-mortems
- Bring more data into regular practices to habitualize feedback



Creating effective feedback loops for AI adoption requires the same foundational elements that Sowmya Subramanian implemented to transform engineering cultures at tech giants like Google and Warner Bros. Discovery. “The first challenge was how to standardize metric definitions. How do you compute mean time to resolution? If it looks different than how another team computes it, then it’s like comparing apples to oranges,” Sowmya explains.

This insight directly applies to AI adoption, where inconsistent or misaligned metrics create noisy signals that prevent AI systems from learning and improving effectively. The implementation of structured rituals can institutionalize these feedback loops throughout an organization. Sowmya recommends tailoring these practices to your organizational needs: “Think of rituals that make sense for your organization. And probably it’s not the same ritual across your entire organization, based on the size of your company and the size of the teams.”

These might include regular model performance reviews, data quality assessments, and cross-functional postmortems for AI adoption when AI systems produce unexpected results. Her innovative “90-5-5 rule” for handling action items

following incidents (90% resolved in the next sprint, 5% within two sprints, and 5% requiring deeper architectural changes) offers a practical framework for addressing AI feedback in a balanced, sustainable way that prevents quick fixes from creating long-term technical debt or unintended consequences. By embedding these practices into your organizational culture, you can make the signal-rich environment necessary for AI systems to learn and improve continuously.



Sowmya Subramanian

Executive Vice President
of Engineering



**WARNER BROS.
DISCOVERY**

Prepare for AI-Driven Software Development

- ☐ Centralize internal knowledge repositories and reduce tribal knowledge gaps
- ☐ Identify repeatable, high-effort tasks with strong APIs as entry points for AI
- ☐ Invest in high-quality, curated training data for internal models
- ☐ Define clear interaction patterns between agents and developers
- ☐ Build platform engineering capabilities to abstract infrastructure complexity
- ☐ Automate guardrails to reduce review fatigue and mental overhead
- ☐ Establish observability and sandbox environments for testing agent workflows
- ☐ Standardize engineering productivity metrics across teams
- ☐ Design AI governance frameworks with auditability and human-in-the-loop policies
- ☐ Institutionalize data-driven rituals (metrics meetings, coaching, postmortems)

Summary

AI-driven development is no longer speculative, it's actively reshaping software engineering from the inside out. The shift toward AI-driven development is underway, but realizing its potential requires more than just integrating another coding tool. The real gains and challenges lie in rethinking how teams work, how systems communicate, and how organizations build trust in automation.

By addressing the six trends outlined in this guide, engineering leaders can ensure their teams are prepared for the future.



Human-agent collaboration

Establishing new patterns for developers and AI to work together



Feedback loops

Creating the signal-rich environments needed for continuous improvement

2025 will belong to those who act boldly, design responsibly, and scale intentionally. The teams that address these challenges today will be positioned to realize the full 10X promise of AI-driven development tomorrow.



Unified knowledge

Building centralized, accessible, and continuously updated knowledge repositories



Modern infrastructure

Creating the technical foundation for agentic workflows



Enabling governance

Designing frameworks that provide guardrails, not barriers



Orchestrated systems

Moving from isolated tools to integrated workflows



About LinearB

LinearB is a developer productivity insight platform designed to enhance developer productivity, streamline workflows, and elevate the developer experience through AI-powered automation and actionable insights. By integrating with existing tools like GitHub, GitLab, and Bitbucket, LinearB provides engineering teams with comprehensive visibility into their development pipelines to identify bottlenecks, optimize resource allocation, and improve software delivery.

[Book a demo](#)



About Dev Interrupted

Dev Interrupted is a newsletter and podcast for software engineering leaders that covers developer productivity, developer experience, and AI-driven software development. Join our community of twenty thousand executives, directors, managers, and more to stay informed on the latest trends in software engineering.

[Follow our Substack](#)